LLM-Based Verilog Adder Generation and Verification Project

A Two-Part Project *for Hardware Design with Large Language Models*

Document Version: October 17, 2025

Table of Contents

- 1. Project Overview
- 2. Prerequisites and Tools
- 3. Available Adder Types
- 4. Part 1: LLM-Based Verilog Generation from Golden Designs
- 4.1. Step 1: Select Two Adders
- 4.2. Step 2: Generate Natural Language Descriptions
- 4.3. Step 3: Generate Verilog Code from Descriptions
- 4.4. Step 4: Manual Verification
- 5. Part 2: LLM-Based Testbench Generation and Simulation
- 5.1. Step 1: Prepare Inputs
- 5.2. Step 2: Generate Testbench with Internal Signal Checking
- 5.3. Step 3: Simulation with Iverilog
- 5.4. Step 4: Analysis and Verification
- 6. Expected Deliverables
- 7. Evaluation Criteria
- 8. Appendix: Sample Workflows

1. Project Overview

This project is designed to explore the capabilities and limitations of Large Language Models (LLMs) in hardware design, specifically in generating and verifying Verilog code for digital adder circuits. The project consists of two interconnected parts that demonstrate the complete workflow from design generation to verification.

Project Objectives

- Understand how LLMs can interpret/generate H/W description language (HDL) code
- Practice reverse engineering Verilog designs into natural language specifications
- Evaluate the accuracy of LLM-generated Verilog code against golden reference designs
- Learn to leverage LLMs for automated testbench generation
- Gain hands-on experience with hardware simulation tools (Iverilog)
- Develop critical analysis skills for comparing synthesizable Verilog architectures

Learning Outcomes

- Ability to analyze and describe digital circuit architectures in natural language
- Understanding of different adder architectures and their trade-offs
- Proficiency in using LLM tools (ChipChat/AutoChip) for hardware design
- Skills in manual code review and verification of HDL designs
- Experience with testbench development and internal signal verification
- Competence in using Iverilog for RTL simulation and debugging

2. Prerequisites and Tools

Required Knowledge

- Basic understanding of digital logic design
- Familiarity with Verilog HDL syntax and semantics
- Knowledge of adder circuits (full adder, half adder, carry propagation)
- Understanding of combinational logic design
- Basic command-line skills for running simulation tools

Required Tools

- Access to LLM tools: ChipChat or AutoChip (for Verilog generation)
- LLM-aided testbench generation tool (enhanced version with internal signal checking)
- Iverilog (Icarus Verilog) - open-source Verilog simulation tool
- Optional: Waveform viewer (GTKWave) for signal analysis

Repository Access

The golden Verilog designs are available in the Verilog-Adders repository. Clone the repository using:

```
git clone https://github.com/FCHXWH823/Verilog-Adders.git
```

3. Available Adder Types

Verilog-Adders repository contains implementations of six different adder architectures. Each adder type has unique characteristics in terms of speed, area, and complexity.

Carry Ripple Adder (RCA)

The simplest adder architecture where carry bits ripple through each stage. Simple to implement but slower for large bit-widths. Files: RCA4.v, RCA8.v

Carry Lookahead Adder (CLA)

Improves speed by pre-computing carry bits using generate and propagate logic. Faster than RCA but requires more logic gates. Files: CLA4.v, CLA8.v

Carry Select Adder (CSA)

Uses parallel computation with two RCAs (assuming carry=0 and carry=1), then selects result. Good balance between speed and area. Files: CSA4.v, CSA8.v

Carry Skip Adder (CSkipA)

Allows carry to skip over blocks when all propagate signals are active. Moderate improvement over RCA with reasonable complexity. Files: CSkipA4.v, CSkipA8.v

Kogge-Stone Adder (KSA)

Advanced parallel prefix adder with logarithmic depth. Fastest architecture but highest area cost. Files: KSA4.v, KSA8.v

Hybrid Adder (HA)

Combines different adder architectures for optimal performance. Complex but efficient for specific use cases. Files: HA8.v

Selection Guidance

For this project, you must select TWO DIFFERENT adder types. Consider choosing:

- One simple architecture (RCA) and one advanced architecture (KSA or CLA) to understand the complexity spectrum
- Two similar architectures (CLA and CSA) to compare different optimization strategies
- Architectures with 8-bit

4. Part 1: LLM-Based Verilog Generation from Golden Designs

In this part, you will use LLMs to understand existing Verilog designs by first converting them to natural language descriptions, then regenerating the Verilog code to verify the LLM's understanding and generation capabilities.

4.1. Step 1: Select Two Adders

Tasks:

- Review the available adder types in Section 3
- Select TWO different adder architectures from the repository
- Ensure you understand the architectural differences between your chosen adders
- Document your selection with justification

Example Selection:

Adder 1: Carry Ripple Adder (RCA8.v) - chosen for its simplicity and educational value
Adder 2: Carry Lookahead Adder (CLA8.v) - chosen to understand optimized carry computation

4.2. Step 2: Generate Natural Language Descriptions

Tasks:

- For each selected adder, read and analyze the golden Verilog code
- Feed the complete Verilog code to your LLM tool
- Prompt the LLM to generate a comprehensive natural language description
- The description should include: architecture overview, module hierarchy, signal flow, key logic operations
- Review and refine the generated description for accuracy and completeness
- Save both descriptions for the next step

Suggested LLM Prompt:

"Analyze the following Verilog code and provide a detailed natural language description of the design. Include: (1) Overall architecture and purpose, (2) Module hierarchy and interfaces, (3) Signal flow and datapath, (4) Key logic operations, (5) Any special design features. Be specific about how the circuit implements its functionality."

4.3. Step 3: Generate Verilog Code from Descriptions

Tasks

- Use ChipChat or AutoChip LLM Verilog generation tool
- Feed the natural language description from Step 2 to the tool
- Instruct the tool to generate Verilog code with the SAME architecture as described
- Ensure the generated code maintains module names, port names, and hierarchical structure
- Save the LLM-generated Verilog code for each adder
- Generate code for both selected adders

Suggested Generation Prompt:
"Based on the following description, generate Verilog code that implements this exact architecture. Maintain *the same module hierarchy, signal names, and design approach described. Use structural Verilog with gate-level primitives where specified."*

4.4. Step 4: Manual Verification

Tasks

- Compare the LLM-generated Verilog code with the golden reference design
- Check for architectural consistency (same module hierarchy, signal flow)
- Verify port declarations (names, widths, directions)
- Compare logic implementation (gate-level vs behavioral, internal structure)
- Identify any differences in implementation details
- Document findings in a comparison report

Verification Checklist:

- Module names match
- Port names and widths match
- Internal signal declarations match
- Module hierarchy is preserved
- Logic implementation is equivalent (functionally)
- Design style matches (structural vs behavioral)
- Any sub-modules are correctly instantiated

Expected Outcomes:
The LLM-generated code may not be identical to the golden design but should be functionally equivalent and maintain the same architectural approach. Document any deviations and assess whether they represent valid alternative implementations or errors.

5. Part 2: LLM-Based Testbench Generation and Simulation

In this part, you will use an enhanced LLM-aided testbench generation tool to create comprehensive testbenches that verify both the functionality and internal signals of your LLM-generated designs.

5.1. Step 1: Prepare Inputs

Actions:

- Gather the natural language description from Part 1, Step 2
- Gather the LLM-generated Verilog code from Part 1, Step 3
- Review the design to identify critical internal signals that should be verified
- List internal signals: carry bits, intermediate sums, propagate/generate signals, etc.
- Prepare these inputs for the testbench generation tool

Internal Signal Identification:

For adders, important internal signals typically include:

- Carry bits between stages (e.g., c[1], c[2], c[3])
- Propagate and generate signals (for CLA, KSA)
- Intermediate sum values (for CSA)
- Block propagate signals (for Skip adder)

5.2. Step 2: Generate Testbench with Internal Signal Checking

Actions:

- Use the enhanced LLM-aided testbench generation tool
- Feed both the natural language description AND the LLM-generated Verilog code
- MODIFY the generation prompt to explicitly request internal signal verification
- Specify which internal signals should be monitored and checked
- Request assertion-based verification or expected value checking
- Generate the testbench code
- Review the generated testbench to ensure it includes internal signal checks

Enhanced Prompt for Internal Signal Checking:

"Generate a comprehensive Verilog testbench for the following design. The testbench should:
1. *Test all input combinations or a representative set of test vectors*
2. *Verify the output signals (sum and carry_out)*
3. *IMPORTANTLY: Monitor and verify internal signals including [list specific signals]*
4. *Include assertions or checks for expected internal signal values*
5. *Report any mismatches between expected and actual values*
6. *Use $display statements to show internal signal values during simulation*
7. *Include a summary of test results at the end"*

Testbench Requirements:

- Must instantiate the DUT (Design Under Test) correctly
- Should include comprehensive test vectors
- Must verify primary outputs (sum, carry_out)
- Must include checks for critical internal signals
- Should use $monitor or $display to show signal values
- Should report PASS/FAIL status for each test
- Should include timing controls (delays) appropriate for simulation

5.3. Step 3: Simulation with Iverilog

Actions:

- Save the LLM-generated testbench as a .v file (e.g., adder_tb.v)
- Ensure the LLM-generated design file is in the same directory or provide correct path
- Compile the design and testbench using Iverilog
- Run the simulation and capture output
- Analyze the simulation results
- If errors occur, debug and potentially regenerate parts of the testbench

Iverilog Commands:

Compilation:

```
iverilog -o adder_sim adder_design.v adder_tb.v
```

Execution:

```
vvp adder_sim
```

Waveform viewing (optional):

```
gtkwave adder_waveform.vcd
```

Debugging Tips:

- If compilation fails, check for syntax errors in the generated code
- Verify module names match between design and testbench instantiation
- Check signal names and widths are consistent
- Use $dumpfile and $dumpvars in testbench to generate VCD waveforms
- Review internal signal values in simulation output to verify correctness
- Compare internal signal behavior with expected values based on architecture

5.4. Step 4: Analysis and Verification

Actions:

- Review simulation output for PASS/FAIL status
- Examine internal signal values during different test cases
- Verify that carry propagation behaves correctly

- Check that intermediate calculations match expected values
- Document any discrepancies between expected and actual behavior
- If issues found, analyze root cause (LLM generation error, testbench error, or design bug)
- Iterate as needed to resolve issues

Analysis Questions:

- Do all test cases pass for the primary outputs (sum, carry_out)?
- Do the internal signals match the expected behavior of the architecture?
- Are there any unexpected signal values or transitions?
- Does the LLM-generated design behave identically to the golden design?
- What differences exist between the LLM design and golden design?
- Are these differences acceptable or do they indicate errors?

6. Part 3: Yosys-Based PPA Optimization Loop

In this part, you will close a feedback loop between an LLM and the open-source synthesis tool Yosys to iteratively discover and refine adder architectures that improve Power, Performance, and Area (PPA). Rather than accepting the first design generated by the LLM, you will use Yosys synthesis reports as objective feedback to guide the LLM toward progressively better architectures. This mirrors state-of-the-art LLM-EDA co-design research and provides experience with automated design space exploration.

Overview: The Optimization Loop

The loop consists of four repeating stages: (1) LLM proposes an adder architecture variant and generates Verilog; (2) Yosys synthesizes the design and reports area, delay, and power estimates; (3) the PPA metrics are fed back to the LLM as context; (4) the LLM proposes a new variant targeting improvements. This loop runs for a configurable number of iterations or until a convergence criterion is met.

Figure 1 (conceptual): LLM Generator → Verilog RTL → Yosys Synthesis → PPA Report → Feedback to LLM → (repeat).

Step 1: Install and Verify Yosys

Yosys is an open-source RTL synthesis framework. It converts Verilog RTL into a gate-level netlist using a standard cell library and reports synthesis statistics including cell area, logic depth, and estimated power.

Installation

On Ubuntu/Debian-based systems:

```
sudo apt-get install yosys
```

Or build from source:

```
git clone https://github.com/YosysHQ/yosys.git
cd yosys && make -j$(nproc) && sudo make install
```

Verify the installation:

```
yosys --version
```

Standard Cell Library Setup

For meaningful area and timing estimates, you need a standard cell library. The FreePDK45 liberty file (.lib) is freely available and suitable for this project:

```
# Download NanGate 45nm Open Cell Library
wget https://raw.githubusercontent.com/The-OpenROAD-Project/OpenROAD-flow-scripts/master/flow/platforms/nangate45/lib/NangateOpenCellLibrary_typical.lib -O nangate45.lib
```

If the above is unavailable, use the built-in Yosys techlibs (no external .lib needed) by replacing the synth target with the generic techmap. The scripts in this section show both approaches.

Step 2: Yosys Synthesis Script and PPA Extraction

Create a reusable Yosys synthesis script that produces a standardized PPA report. Save the following as synth_adder.ys:

```
# synth_adder.ys  -- run with: yosys synth_adder.ys
read_verilog $env(ADDER_FILE)
hierarchy -check -top $env(TOP_MODULE)
proc; opt; fsm; opt; memory; opt
```

```
techmap; opt
dfflibmap -liberty nangate45.lib
abc -liberty nangate45.lib -constr constraints.sdc
clean
stat -liberty nangate45.lib
tee -o timing_report.txt sta
```

A Python wrapper parses the Yosys stat output into structured PPA metrics. Save the following as run_yosys.py:

```
import subprocess, re, os, json

def synthesize(verilog_file, top_module, lib_file='nangate45.lib'):
    env = os.environ.copy()
    env['ADDER_FILE']  = verilog_file
    env['TOP_MODULE']  = top_module
    result = subprocess.run(
        ['yosys', '-s', 'synth_adder.ys'],
        capture_output=True, text=True, env=env)
    return parse_stats(result.stdout + result.stderr)

def parse_stats(log):
    ppa = {}
    # Area: total cell area in um^2
    m = re.search(r'Chip area for.*?:\s+([\d.]+)', log)
    ppa['area_um2'] = float(m.group(1)) if m else None
    # Cell count
    m = re.search(r'Number of cells:\s+(\d+)', log)
    ppa['cell_count'] = int(m.group(1)) if m else None
    # Logic levels (depth)
    m = re.search(r'Longest topological path.*?\((\d+) levels?\)', log)
    ppa['logic_levels'] = int(m.group(1)) if m else None
    return ppa

if __name__ == '__main__':
    import sys
    ppa = synthesize(sys.argv[1], sys.argv[2])
    print(json.dumps(ppa, indent=2))
```

Test the script on a known design:

```
python run_yosys.py RCA8.v RCA8
```

You should see a JSON object with area_um2, cell_count, and logic_levels. Record these as your baseline.

Step 3: LLM Architecture Proposal Loop

The core of the optimization is a Python script that orchestrates the LLM-Yosys loop. Save the following as optimize_adder.py:

```
import json, os, re, subprocess
from openai import OpenAI   # or use anthropic SDK

client = OpenAI(api_key=os.environ['OPENAI_API_KEY'])

SYSTEM_PROMPT = """You are an expert digital circuit designer.
Your goal is to generate synthesizable Verilog for an 8-bit adder
```

```
that minimizes area (cell_count) while meeting a delay target
of at most 12 logic levels. Respond with ONLY valid Verilog code,
no markdown fences, no commentary."""

def llm_generate(history):
    response = client.chat.completions.create(
        model='gpt-4o',
        messages=[{'role':'system','content':SYSTEM_PROMPT}] + history)
    return response.choices[0].message.content.strip()

def build_feedback_prompt(iteration, ppa, prev_code, best_ppa):
    return f"""Iteration {iteration} synthesis results:
  - Area (cells): {ppa['cell_count']}
  - Logic levels (delay): {ppa['logic_levels']}
  - Area (um^2): {ppa['area_um2']}

Best so far: {best_ppa['cell_count']} cells, {best_ppa['logic_levels']}
levels.

Propose a new 8-bit adder Verilog implementation that reduces cell
count while keeping logic_levels <= 12. Consider architectural
changes such as: partial prefix trees, hybrid CLA+RCA, Brent-Kung
tree, or carry-select grouping. Provide only Verilog code."""

def run_loop(baseline_verilog, top='adder8', max_iter=10):
    history = []
    best_ppa = {'cell_count': 1e9, 'logic_levels': 1e9, 'area_um2': 1e9}
    best_code = baseline_verilog
    results = []

    # Seed: ask LLM to reproduce the baseline architecture
    history.append({'role':'user',
        'content': f'Here is an 8-bit adder
baseline:\n{baseline_verilog}\n'
                   'Start from this design. Your first proposal may be
identical.'})

    for i in range(1, max_iter + 1):
        print(f'\n=== Iteration {i} ===')
        verilog = llm_generate(history)

        # Write to file and synthesize
        fname = f'candidate_{i}.v'
        with open(fname, 'w') as f: f.write(verilog)
        try:
            ppa = synthesize(fname, top)
        except Exception as e:
            print(f'  Synthesis failed: {e}')
            history.append({'role':'assistant','content':verilog})
            history.append({'role':'user',
                'content':'Synthesis failed (likely invalid Verilog). '
                           'Fix all syntax errors and resubmit.'})
            continue
```

```
        print(f'  Cells: {ppa["cell_count"]}  Levels:
{ppa["logic_levels"]}  Area: {ppa["area_um2"]:.1f} um^2')
        results.append({'iteration': i, 'ppa': ppa, 'file': fname})

        # Update best
        if (ppa['logic_levels'] is not None and ppa['logic_levels'] <= 12
                and ppa['cell_count'] < best_ppa['cell_count']):
            best_ppa = ppa
            best_code = verilog
            print('  *** New best! ***')

        # Build next prompt with feedback
        history.append({'role':'assistant','content':verilog})
        history.append({'role':'user',
            'content': build_feedback_prompt(i, ppa, verilog, best_ppa)})

    return best_code, best_ppa, results

if __name__ == '__main__':
    with open('RCA8.v') as f: baseline = f.read()
    best_v, best_ppa, log = run_loop(baseline, max_iter=10)
    with open('best_adder.v', 'w') as f: f.write(best_v)
    with open('optimization_log.json', 'w') as f:
        json.dump({'best_ppa': best_ppa, 'iterations': log}, f, indent=2)
    print(f'\nBest: {best_ppa}')
```

Running the Loop

Execute the optimization loop with:

```
export OPENAI_API_KEY=your_key_here
python optimize_adder.py
```

The loop will run up to 10 iterations (configurable via max_iter). At each iteration it prints the PPA metrics, tracks the best-seen design, and appends the synthesis feedback to the conversation history before generating the next candidate.

Step 4: Constraints and Optimization Targets

Yosys ABC synthesis accepts optional timing constraints. Create a constraints.sdc file to set a target clock period (in ns) which ABC uses to optimize for timing:

```
# constraints.sdc
create_clock -name clk -period 2.0
set_input_delay  0.2 -clock clk [all_inputs]
set_output_delay 0.2 -clock clk [all_outputs]
```

Adjust the clock period based on your target. A tighter period forces ABC to prioritize delay reduction, which the LLM feedback prompt should reflect. For this project, use the following three optimization modes and run the loop once per mode:

- Mode A – Area minimization: max_iter=10, delay target = 14 logic levels (relaxed)
- Mode B – Delay minimization: max_iter=10, delay target = 6 logic levels (tight)
- Mode C – Balanced: max_iter=10, delay target = 10 logic levels, minimize cells

Step 5: Analyzing and Comparing Optimization Trajectories

After each loop run, parse optimization_log.json and plot the PPA trajectory. Save the

following as plot_ppa.py:

```
import json, matplotlib.pyplot as plt

with open('optimization_log.json') as f: log = json.load(f)
iters  = [r['iteration']          for r in log['iterations']]
cells  = [r['ppa']['cell_count']  for r in log['iterations']]
levels = [r['ppa']['logic_levels'] for r in log['iterations']]

fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True, figsize=(8, 5))
ax1.plot(iters, cells,  marker='o', color='steelblue')
ax1.set_ylabel('Cell count (area)'); ax1.grid(True)
ax2.plot(iters, levels, marker='s', color='firebrick')
ax2.set_ylabel('Logic levels (delay)'); ax2.set_xlabel('Iteration')
ax2.grid(True)
plt.suptitle('LLM-Yosys Optimization Trajectory')
plt.tight_layout(); plt.savefig('ppa_trajectory.pdf'); plt.show()
```

Compare trajectories across the three optimization modes. Analyze: (1) how quickly the LLM finds improvements; (2) whether it converges or oscillates; (3) which architectural changes produced the largest PPA gains; (4) how the LLM-discovered architecture compares to the known classical adders from Part 1.

Step 6: Architecture Identification and Explanation

For each best design discovered by the loop, prompt the LLM to explain the architectural decisions it made:

"Analyze the following Verilog adder design that was discovered through automated synthesis-guided optimization. Explain: (1) what adder architecture or hybrid architecture this represents; (2) why this structure reduces cell count compared to a Ripple Carry Adder; (3) what are the trade-offs in delay and fanout; (4) how it compares to Kogge-Stone, Brent-Kung, and Carry Select adders."

Document the LLM's architectural explanation alongside the Yosys PPA numbers. This analysis forms a key part of your deliverable and demonstrates understanding of the LLM's design reasoning.

Step 7: Verification of the Optimized Design

The best design from the loop must be functionally verified before it is accepted. Run the following verification flow:

Functional Simulation

Reuse the testbench generation workflow from Part 2 to create a testbench for the optimized adder. Simulate with Iverilog:

```
iverilog -o opt_sim best_adder.v best_adder_tb.v
vvp opt_sim
```

Equivalence Checking with Yosys

Use Yosys' built-in equivalence checking to formally verify that the optimized adder is functionally identical to the golden reference design:

```
# equiv_check.ys
read_verilog -sv RCA8.v
rename RCA8 gold
read_verilog -sv best_adder.v
rename adder8 gate
```

```
equiv_make gold gate
equiv_simple
equiv_status
```

Run with:

```
yosys -s equiv_check.ys
```

A successful equivalence check guarantees that PPA improvements did not come at the cost of functional correctness. If the check fails, investigate whether the LLM introduced a design error (e.g., incorrect carry logic) or a valid micro-architectural change that changes the interface (e.g., registered outputs).

Step 8: Multi-Start Exploration

A single optimization run may converge to a local optimum. To improve coverage of the design space, run the loop from multiple different starting architectures. Modify optimize_adder.py to accept a starting architecture argument:

```
python optimize_adder.py --baseline KSA8.v  --mode area  --out ksa_area/
python optimize_adder.py --baseline CLA8.v  --mode delay --out cla_delay/
python optimize_adder.py --baseline RCA8.v  --mode balanced --out
rca_bal/
```

Compare the final PPA of the best designs across all runs. Summarize in a table with columns: Starting Architecture, Optimization Mode, Final Cells, Final Logic Levels, Final Area (um^2), Architecture Identified.

7. Expected Deliverables

Upon completion of this project, you should submit the following deliverables:

Adder Selection Document

A document explaining which two adders you selected and why. Include architectural differences.

Natural Language Descriptions

Complete natural language descriptions for both selected adders, generated in Part 1, Step 2.

LLM-Generated Verilog Designs

The Verilog code files generated by ChipChat/AutoChip for both adders (Part 1, Step 3).

Verification Report (Part 1)

A detailed comparison between LLM-generated code and golden reference designs. Include similarities, differences, and assessment of equivalence.

LLM-Generated Testbenches

Complete testbench files with internal signal checking for both adders (Part 2, Step 2).

Simulation Results

Iverilog simulation output showing test execution and results. Include screenshots or captured console output.

Analysis Report (Part 2)

Analysis of simulation results, including verification of internal signals, any issues encountered, and how they were resolved.

Yosys Synthesis Scripts and Configuration (Part 3)

All Yosys scripts (synth_adder.ys, equiv_check.ys, constraints.sdc), the run_yosys.py wrapper, and the optimize_adder.py orchestration script. Include the nangate45.lib or equivalent liberty file used.

Optimization Log and PPA Trajectories (Part 3)

The optimization_log.json files for all three optimization modes (area, delay, balanced). The PPA trajectory plots (ppa_trajectory.pdf) for each run, and the Multi-Start comparison table summarizing final PPA across all starting architectures and modes.

Best Adder Designs and Verification Report (Part 3)

The best_adder.v Verilog files from each optimization run. Iverilog simulation output confirming functional correctness. Yosys equivalence checking output (PASS/FAIL). LLM architectural explanation for each best design (Step 6 output).

Final Project Report

A comprehensive report discussing:

- Your experience with LLM-based hardware design
- Accuracy and limitations of LLM-generated code
- Quality of LLM-generated testbenches
- Lessons learned and best practices
- Recommendations for using LLMs in hardware design

8. Evaluation Criteria

Your project will be evaluated based on the following criteria:

Completeness (25%)

- All required steps completed for both adders
- All deliverables submitted
- Documentation is thorough and well-organized

Technical Accuracy (30%)

- Natural language descriptions accurately reflect the Verilog designs
- LLM-generated Verilog is functionally correct
- Testbenches properly verify both outputs and internal signals
- Simulation results are correctly interpreted

Analysis Quality (25%)

- Verification reports show deep understanding of designs
- Differences between LLM and golden designs are well analyzed
- Simulation results are thoroughly examined
- Critical thinking applied to LLM capabilities and limitations

Problem-Solving (10%)

- Ability to debug and resolve issues during the project
- Effective iteration on prompts to improve LLM outputs
- Appropriate modifications to generated code when necessary

Documentation (10%)

- Clear and professional writing
- Proper formatting of code and results
- Logical organization of materials
- Inclusion of relevant examples and evidence

9. Appendix: Sample Workflows

Sample Workflow for RCA (4-bit Ripple Carry Adder)

Part 1 Sample

1. Selected Adder: RCA4.v from Carry Ripple Adder directory

2. **Generated Description (excerpt):**

"This design implements a 4-bit Ripple Carry Adder using four full adder modules. The architecture consists of a Full Adder (FA) module and the top-level RCA4 module. Each full adder computes one bit of the sum and generates a carry output that feeds into the next stage. The carry ripples from LSB to MSB, with the first stage receiving a carry-in of 0..."

3. LLM Generation: Feed description to ChipChat

4. Verification: Compare generated code structure, verify FA module definition, check carry chain

Part 2 Sample

1. Prepare natural language description and LLM-generated RCA4 design

2. Identify internal signals: c[1], c[2], c[3] (carry bits between stages)

3. Prompt for testbench generation with internal checks

4. Simulate with iverilog

5. **Sample output:**

```
Test 1: a=0001, b=0001, expected_sum=0010, expected_c1=0, expected_c2=0, expected_c3=0
Result: PASS - sum=0010, cout=0, internal carries correct
Test 2: a=1111, b=0001, expected_sum=0000, expected_c1=1, expected_c2=1, expected_c3=1
Result: PASS - sum=0000, cout=1, internal carries correct
```

Tips and Best Practices

- Start with simpler adders (RCA) before attempting complex ones (KSA)
- Be specific in your prompts to the LLM - include architectural details
- When comparing code, focus on functional equivalence rather than exact match
- Test edge cases: all zeros, all ones, carry propagation across all bits
- Use waveform viewers to visualize internal signal behavior
- Iterate on your prompts if initial LLM output is unsatisfactory
- Keep detailed notes throughout the process for your final report
- Don't hesitate to manually fix minor syntax errors in generated code
- Verify that internal signals make logical sense based on the architecture
- Compare simulation results between LLM design and golden design if possible

Common Challenges and Solutions

LLM generates behavioral code instead of structural

Solution: Explicitly request structural/gate-level implementation in your prompt. Provide examples of desired style.

Testbench doesn't include internal signal checking

Solution: Be very explicit in your prompt. List specific signal names and what should be checked.

Simulation fails with compilation errors

Solution: Check for syntax errors, verify module names match, ensure all signals are declared.

Internal signal values seem incorrect

Solution: Trace the logic manually for simple test cases. Use waveform viewer to see signal transitions.

LLM-generated code has different module hierarchy

Solution: This may be acceptable if functionally equivalent. Document the differences and assess impact.

Conclusion

This project provides hands-on experience with the emerging field of LLM-assisted hardware design. By working through both the generation and verification workflows, you will gain insights into the capabilities and current limitations of AI tools in the hardware domain. The skills developed in this project - from architectural analysis to verification methodologies - are valuable for modern digital design practices.

Remember that LLMs are powerful tools but require careful validation. Always verify generated designs through simulation and, ideally, formal verification methods. Use these tools to augment your expertise, not replace your critical thinking and domain knowledge.

Good luck with your project!